Unit-4

SCHEMA REFINEMENT AND NORMAL FORMS

4.1 Schema Refinement

Relational schema design.

- Two levels of considering the effectiveness of schema:
  - **Logical level:** whether a schema has intuitive appeal for users.
  - **Manipulation level:** whether it makes sense from an *efficiency* or *correctness* point of view.

4.2 Functional Dependencies and Normalization

- Guidelines for database schema design: how to decide on a particular schema?

  For example: Consider these two schemas for the same database:

  Schema *S1:*

  EMPLOYEE (LANME, FNAME, SSN, DNO)

  DEPT (DNUM, DNAME, MGRSSN)

  Schema *S2:*

  EMPLOYEE (LANME, FNAME, SSN, DNO, DNUM, DNAME, MGRSSN)

  Which one is better?

  - Informal and formal methods.
  - **Formal methods:** involve the notion of *functional dependency.*

Informal Guidelines

Try to make user interpretation easy.

Compare

- Relational Schema *S1:*

  EMP (FNAME, LNAME, SSN)
  WORKS_ON (SSN, PNO)
  PROJECT_LOC (PNO, PLOC)

- Relational Schema *S2:*

  EMP (FNAME, LNAME, SSN, PNO, PLOC)

Perhaps S2 has too much information (to absorb) per tuple.

Try to reduce redundancy.

- In schema S2 above, suppose there are only a few projects

  $\Rightarrow$ PLOC is unnecessarily repeated too often.
- On the other hand, S1repeats SSN in WORKS_ON

  $\Rightarrow$ But SSN is a smallest attribute than PLOC (which may be large string).

Try to avoid update anomalies.

- Consider the schemas:
  - Schema *S1:*

EMPLOYEE (ENAME, SSN, BDATE, ADDR, DNO)
DEPT (DNUM, MGRSSN, DNAME)
WORKS_ON (SSN, PNO, HOURS)
PROJECT (PNUM, PLOCATION)

- Schema *S2:*
  EMP_DEPT (ENAME, SSN, BDATE, ADDR, DNO, DNAME, MGRSSN)
  EMP_PROJ (SSN, PNUM, HOURS, ENAME, PNAME, PLOCATION)
  Both have the same attributes. Unfortunately, S2 can create problems called *update anomalies.*

Insertion anomalies
Consider inserting "John Smith works in Dept. 5", i.e. <John, Smith, 123456789, … , 5, <dname>, <mgrssn>>.
Every time a tuple of this sort is entered, we have to check that DNAME is correct

We have to scan the whole relation (worst-case).
Consider creating a new department: DNO = 9, DNAME = "Sales"
Only one way to insert this information

Create NULL values for employee information
But that means a NULL primary key value (SSN)!.

Deletion anomalies
If we delete the last employee in the 'Research' department e.g <John, Smith, 123456789, … , 5, 'Research', …>
Then we will lose the information "Department 5 is the Research Department".

Modification anomalies
Suppose we change the manager of department 5.
We have to change MGRSSN for all department 5 employees.
Full scan of database.
Thus Schema S2 has many problems. On the other hand:
S1 – has 4 relations
S2 – has 2 relations
For many queries, we will need more joins using S1
SQL code with S1 will be more complicated because of the extra joins.
(One Solution: use S1 but create views based on needed joins)

Try to avoid too many NULL values.
This may occur in 'fat' relations (with too many attributes).
Space is wasted.
Problems occur when using aggregate functions like **count** or **sum.**
NULLs can have different intentions:
The attribute does not apply
Values is unknown, and will remain unknown

Value is unknown at present.

☞ To avoid the above problems we need a theory of schema design. i.e. *functional dependencies* and *normalization.*

4.3 Functional Dependencies

First, some convenient notions (and notation):
Suppose our relational database has attributes $A_1, \ldots, A_n$.

Let R denote the schema $R = (A_1, \ldots, A_n)$.

Typically, of course, we will have several relations,

e.g., EMP (A_1, A_3, A_9), DEPT (A_9, A_7, A_8) ... etc.

- For convenience, let us consider a relation with all the attributes i.e. with schema $R = (A_1, A_2, \ldots, A_n)$.

Definition: A **functional dependency** (FD) between two sets of attributes X and Y, denoted by $\mathbf{X \rightarrow Y}$, specifies a certain relationship or connection between X and Y. Specifically, it says:
If t_1 and t_2 are any two tuples in any instance of R such that

$$t_1[X] = t_2[X]$$

then

$$t_1[Y] = t_2[Y]$$

Intuitively, $X \rightarrow Y$ means: if you know the X-values of a tuple, then that uniquely determines the Y-values.

Note: X and Y can be single attributes or *groups* of attributes.

Example: Consider the relational database schema:
(SSN, PNUMBER, ENAME, PNAME, PLOCATION)

Suppose X = {SSN}

Y = {ENAME}

Then $X \rightarrow Y$, 1.e., SSN uniquely determines ENAME

From the definition, if we are given two tuples t_1 and t_2, e.g.,

t_1 = <123456789, …, Smith, …>

t_2 = <123456789, …, Smith, …>

where

$t_1[X] = t_2[X]$ (i.e. $t_1[SSN] = t_2[SSN]$)

then it must be true that

$t_1[Y] = t_2[Y]$ (i.e. t_1[ENAME] = t_2[ENAME])

Thus, we can't have two tuples with the same SSN and different ENAMES's. Similarly, the functional dependency

{SSN, PNUMBER} → {PNAME, PLOCATION}

is a reasonable assumption or a reasonable constraint to declare.

Functional dependencies are specified to capture dependencies for all instances.

It may be just happen that project locations (PLOCATIO) are <u>all</u> different for a particular instance

⇒ we may led believe that {ENAME} → {PLOCATION}.

But, later on 'Smith' might work on projects in different locations

⇒ this would be poor choice of a FD.

A FD is a property of the meaning of attributes.

⇒ it should hold for all possible instances.

FD's for specific relations.

Although we have defined FD's on the universe of attributes, we will often discuss FD's within particular relations.

For Example, the database schema might be:

(SSN, PNUMBER, ENAME, PNAME, PLOCATION).

We might have the relation

WORKS_ON (SSN, PNUMBER, ENAME, PNAME).

Here, we can identify the FD {SSN} → {ENAME}

in WORKS_ON.

- *Sometimes a diagram is used to show FD's*

Sets of Functional Dependencies

- Consider the following example:

EMP_DEPT (ENAME, SSN, BDATE, ADDRESS, DNUMBER, DNAME, DMGRSSN)

The obvious FD's are:

{SSN} → {ENAME, BDATE, ADDRESS, DNUMBER}

{DNUMBER} → {DNAME, DMGRSSN}

Let F denote the above collection of FD's

From F, we can infer

{SSN} → {DNAME, DMGRSSN}.

Why? Because, SSN uniquely determines DNUMBER and DNUMBER uniquely determines {DNAME, DMGRSSN}.

Also, the following FD's are examples of trivial FD's inferred form F.

{SSN} → {SSN}

[SSN, DNUMBER} → {DNUMBER}

- **Definition.** Suppose *F* is a set of FD's. Then, F^+, the **closure** of *F* is the set of FD's that includes *F* and all the FD's that can be inferred from *F*.
  Inference Rules for FD Sets

- *NOTE the following:*
  1. When we say X →Y, X and Y are subsets of the universe of attributes.
  2. For convenience, we will sometimes drop the set notation and commas within sets. Suppose F is the set of FD's:

     X →Y,Y → Z.

     Then, from *F* we can infer: X →YZ.

     Here, *YZ* denotes the union of *Y* and *Z*.

     For example, *F* is

     X = {SSN} → {ENAME} = *Y*

     X = {SSN}→ {BDATE} = Z.

     From which we conclude

     X = {SSN} → {ENAME, BDATE} = YUZ = YZ

From above, we can devise a rule: if X → Y and X →Z, then X→YZ.
Such a rule is called in **inference rule.**

Standard inference rules for FD's:

1. **Reflexive rule.** If Y ⊆X then X→Y.

e.g. {SSN, BDATE} → {BDATE}

2. **Augmentation rule.** If X→Y then XZ→YZ for any group of attributes Z.

X = {SSN}

Y = {BDATE}

Z = {PLOCATION}

Then, {SSN} → {BDATE} implies

{SSN, PLICATION} → {BDATE, PLOCATION}

3. Transitive rule. If X → Y and X →Z, then X→Z.

e.g., the FD's

{SSN} → {DNUMBER}

{DNUMBER} → {DNAME}

together imply

{SSN} → {DNAME}

4. Decomposition rule. If X→Y YZ then X→Y.

e.g. the FD

{SSN} → {ENAME, DNUMBER}

implies {SSN} → {DNUMBER}

5. Union rule. If X→Y and X→Z then X →YZ.

e.g. the FD's

{SSN} → {ENAME}

{SSN} → {BDATE}

imply {SSN} → {ENAME, BDATE}.

6.Pseudo transitive rule. If X →Y and WY → then WX → Z.

e.g. the FD's

{PNO} → {PNAME}

{SSN, PNAME} → {HOURS}

imply

{SSN,PNO} → {HOURS}

- One cane use Rules 1-6 to determine $F+$.
- It turns out that Rules 1-3 are sufficient to completely determine F^+.

Rules 1-3 are called **Armstrong's Rules** in honor of the person who proved this result.

- *Equivalent FD sets.*
  - For most FD-sets F, the closure F^+ is probably quite large.
  - While we are interested in the theoretical implications of F^+ for any F, we are rarely going to compute F^+.
  - $\Rightarrow$ Working with F turns out to be good enough.
  - **Definition.** FD-sets E and F are **equivalent** if $E^+ = F^+$, i.e., if their closures are equal.
  - If E is a set of FD's smaller than F and yet $E^+ = F^+$, then it is easier to work with E.
  - If $E^+ = F^+$ we also say that E **covers** F or is a **cover for** F.

Attribute Set Closures

- If X is an attribute set, we are often interested in all attributes (functionally) determined by X. i.e., what is the largest Y for which $X \rightarrow Y$?

- **Definition.** If F is a set of FD's and X is a set of attributes, then X^+ is the **closure of X under** F if X^+ is the largest set of attributes functionally determined by X using inference rules on FD's in F.
- **Algorithm for determining X^+.**

```
Algorithm: COMPUTE  X+
Input: X, F.
Output: X+.
1. X+ ← X;
2. repeat
3.         old_X+ ← X+;
4.       for each FD Y → Z in F do
5.          if Y ⊆ X+ then
6.             X+ ← X+ U Z;
7. until old_ X+ = X+;
8. return X+;
```

{PNUMBER} → {PNAME, PLOCATION}

{SSN, PNUMBER} → {HOURS}

and suppose we want the closure of X= {SSN,PNUMBER}under F.

Initially, $X^+ \leftarrow$ {SSN, PNUMBER}.

In the first iteration of the outer loop, old_ X^+= {SSN, PNUMBER}.

Then, there are 3 iterations of the inner loop (one for each FD). This gives us

X^+ = {SSN, PNUMBER, ENAME, PNAME, PLOCATION}.

The second iteration of the outer loop creates no changes.

$\Rightarrow$ algorithm terminates with

$X^+ = \{SSN, PNUMBER, ENAME, PNAME, PLOCATION\}$.

4.5 Normal Forms

- Normal forms are properties of relations.
- There are many normal forms:

1NF - First Normal Form

2NF - Second Normal Form

3NF - Third Normal Form

  - We say a relation is **in** xNF if its attributes satisfy certain properties (via their FD's)
  - Generally, these properties are desirable.
  - For example, if we desire the 3NF for a relational database:
    - We will test the relations in the database to see which are in 3NF.
    - Those that are not in 3NF will be decomposed into smaller relations (smaller numbers of attributes) until we have each relation satisfying the 3NF properties.
  - In the real world, most people achieve 3NF. It is slightly better to achieve BCNF (Boyce-Codd), but 3NF is considered "not bad".
  - The end result is: if a database is in BCNF (or 3NF), many anomalies are avoided.
  - FD's were designed to test for Normal Forms.

Normal Forms: Primary Key Version

- It is easier to understand normal forms by first considering a simpler version – normal forms of primary keys.
- Recall definitions and notation:
  - Let R denote a relation schema.
  - A **superkey** $S \subseteq R$ can be used to uniquely determine tuples.
  - A **key** is a minimal superkey.
  - A **candidate key** - same as a key. Used when several keys are under consideration.
  - A **Primary key** - a **key** designated for common use.
  - A **prime attribute** – an attribute belonging to some candidate key (not necessarily the primary key).
  - A **nonprime attribute –** not belonging to any candidate key.

■ **1NF: First Normal Form**

- **Definition.** A relation is in 1NF if:
  1. the value of any attribute in any tuple is a single value, and

2. domains of attributes contain only atomic values.

- Example of relations not satisfying 1NF (Fig 12.8(b));

  $\Rightarrow$ Does not satisfy first part of definition. (Fig 12.9 b);

  $\Rightarrow$ Does not satisfy second part of definition

  (Contains nested relations).

- It is easy to transform the above relations to satisfy 1 NF by:
  1. adding tuples or
  2. creating new relations

  Note: we could have created additional tuples but this would have required more space.

- 1NF is nowadays taken for granted (i.e., later formulations of relational theory assume relations to be in 1NF).

  $\Rightarrow$ We will assume all relations are in 1NF.

■ **2NF: Second Normal Form**

- **Definition.** A FD $X \rightarrow Y$ is a **partial dependency** if there exists an attribute A ε X such that

  $X - A \rightarrow Y$.

  We say that Y is **partially dependent** on X.

  Example: consider the relation

  EMP_PROJ (<u>SSN, PNUMBER</u>, HOURS, ENAME, PNAME, PLOCATION).

  Here we can identify some FD's such as

  {SSN,PNUMBER} $\rightarrow$ {HOURS}

  {SSN,PNUMBER} $\rightarrow$ {ENAME}

  Now, neither one of

  {SSN} $\rightarrow$ {HOURS}

  {PNUMBER} $\rightarrow$ {HOURS}

  is true.

Thus, {SSN,PNUMBER} $\rightarrow$ {HOURS} is not a partial dependency.

But, {SSN} $\rightarrow$ {ENAME}.

Hence, ENAME is partially dependent on the primary key {SSN, PNUMBER}.

- **Definition.** A relation schema is in 2NF if no nonprime attribute is partially dependent on the primary key.
- Thus, in the above example EMP_PROJ is not in 2NF.
  (It is, however, in 1NF).

- Why is this a problem?

  Suppose an instance looks like:

SSN	PNUMBER	HOURS	ENAME	PNAME	PLOCATION
123456789	1	10	Smith	Pepsi	New York
987654321	2	15	Jones	Coke	Atlanta

  Suppose we want to insert the tuple

  < 987654321,3,12, Jones, Sprite, Atlanta>.

  Because we have the FD: {SSN} $\rightarrow$ {ENAME} we will have to check that

  < 987654321,...,Jones,…>

  is valid, i.e., that it matches other SSN, ENAME values for Jones.

  $\Rightarrow$ we have to scan the whole relation (worst-case) to check

  $\Rightarrow$ insertion anomaly.

  Similarly, if project # 1 is deleted:

  $\Rightarrow$ we will lose the information "123456789 is the SSN of Smith"

  $\Rightarrow$ deletion anomaly.

  This relation also has a modification anomaly:

  $\Rightarrow$ if Smith changes name to Brown

  $\Rightarrow$ have to propagate change to all relevant tuples.

  To solve the problem, consider the decomposition of

  EMP_PROJ (<u>SSN, PNUMBER</u>, HOURS, ENAME, PNAME, PLOCATION).

  into

  EP1 (SSN, PNUMBER, HOURS)

EP2 (SSN, ENAME)

EP3 (PNUMBER, PNAME, PLOCATION)

Those relations are in 2NF, with the important FD's:

{SSN, PNUMBER} → {HOURS}

{SSN} → {ENAME}

{PNUMBER} → {PNAME,PLOCATION}

There are no partial dependencies of nonprime attributes on primary keys.

■ 3NF: Third Norma Form

- **Definition.** The FD X→Y is a **transitive dependency** in relation R if there exists a set of attributes Z in R such that

  1. X→Y and Z →Y
  2. Z is not a subset of any key of R

  Example: consider the relation

  EMP _DEPT(NAME,SSN,BDATE,ADDRESS,DNO,DNAME,MGRSSN).

  - Observe that EMP_DEPT is in 2NF since no partial dependencies exist at all (and hence partial dependencies on the primary key don't exist).
  - Next , we have FD's:

{SSN} → {DNO}

{DNO} → {MGRSSN}

Note that DNO is not part of any key. Thus, there is a transitive dependency of MGRSSN on SSN.

- **Definition.** A relation R is in 3NF if
  1. it is in 2NF and,
  2. no nonprime attribute is transitively dependent on the primary key.

In the above example:

- EMP _DEPT is in 2NF
- MGRSSN is a nonprime attribute transitively dependent on the primary key SSN.

⇒ EMP _DEPT is not in 3NF

- Why should we care about 3NF?

Consider the following instance of EMP _DEPT:

ENAME	SSN	BDATE	ADDRESS	DNO	DNAME	MGRSSN
Smith	…	…	…	5	Research	111111111
Jones	…	…	…	6	Sales	444444444

Suppose we insert the tuple <Brown….,5, Research, 333333333>

⇒ we would have to check that the DNO matches the MGRSSN (wrong in this case) ⇒ scanning the database ⇒ insertion anomaly.

Similarly, if Smith's DNO changes to 6

⇒ We will have to also insert the correct MGRSSN ⇒ MODIFICATION ANOMALY.

A deletion anomaly also occurs, if we delete Smith's tuple and Smith is the last Research employee.

⇒ We will lose the information "Dept#6is Research".

- To solve a 3NF problem, we can decompose relations.

In the above example:

ED1 (ENAME, SSN, BDATE, ADDRESS, DNO)

ED2 (DNO, DNAME, MGRSSN)

Note:

1. ED1 and ED2 are in 3NF
2. EMP _DEPT can be recovered by joining ED1 AND ED2.
3. The join will not create spurious tuples.

General Definitions of 2NF and 3NF

- We have defined 2NF and 3NF using primary keys.
- General definitions based on any candidate key are desirable.
- 2NF:

- *Primary version:* A relation schema R is in 2NF if every nonprime attribute A in R is not partially dependent on the Primary key.
- *General version:* A relation schema R is in 2NF if very nonprime attribute A in R is not partially dependent on any key of R

Consider the following example:

LOTS (PROP_ID, COUNTY, LOT #, AREA, PRICE, TAX_RATE)

Here,

- The relation represents statewide information about properties, their areas (sizes), price and tax rates assessed.
- PROP _ID is the primary key.
- LOT # is the code used within a county (by the county government).

$\Rightarrow$ (COUNTRY, LOT#) is a candidate key.

Suppose tax rates are set by country, i.e., we have the partial FD

$$\{\text{COUNTY}\} \rightarrow \{\text{TAX_RATE}\}.$$

Then, LOTS is in 2NF according to the primary version of the definition (no partial dependency on the primary key).

But the partial dependency on COUNTY causes the general definition to fail.

$\Rightarrow$ LOTS is not in 2NF.

- We want the general definition to hold, because otherwise we will have to check the FD {COUNTY}$\rightarrow$ {TAX _RATE} for every insertion.

We can use the decomposition:

LOTS1 (PROP _ID, COUNTY, LOT#, AREA, PRICE)

LOTS2 (CONUTY, TAX _RATE)

- Thus we see how our elaborate definitions of normal forms help us catch problems in seemingly innocuous schemas (like LOTS).

- **3NF:**
  - *Primary version:* A relation R is in 3 NF if
    1. it is in 2NF , and
    2. no nonprime attribute is transitively dependent on the primary of R.
  - *General version:* A relation R is in 3NF if

1. it is in 2NF, and
2. no nonprime attribute is transitively dependent on <u>any key</u> of R.

Now observe this:

- If $X \subseteq \{A1,\ldots,An\}$ is any key then $X \rightarrow Y$ for any $Y \subseteq \{A1,\ldots,An\}$
- Suppose for some relation R
  1. X is the primary key.
  2. Y is some other key.
  3. Z is transitively dependent on Y, i.e., there are FD's
     $Y \rightarrow W$ and $W \rightarrow Z$.

  But $X \rightarrow W$ since X is a key.

  $\Rightarrow$ Z is transitively dependent on X (the primary key)

  $\Rightarrow$ the two 3NF definitions are identical.

■ **BCNF: Boyce-Codd Normal Form**

- Consider the relation ADDR _INFO (CITY, ADDRESS, ZIP).
  [Here, ADDRESS denotes street address only].

  We can identify the natural FD's.

  {CITY, ADDRESS} $\rightarrow$ {ZIP}

  {ZIP} $\rightarrow$ {CITY}

  Possible keys are: {CITY, ADDRESS} or {ADDRESS, ZIP}.

  Is ADDR_INFO in 2NF?

  - Recall: no nonprime attribute should have a partial dependence on a key.
  - Here, there are no nonprime attributes

- It passes the 2NF test

  Is ADDR _INFO in 3NF?

  - It is in 2NF.
  - Recall: we should not have nay transitive dependence on a key.
  - When considering {CITY, ADDRESS} as a key, there is only on attribute left (ZIP)

  $\Rightarrow$ can't have a transitive dependency.

  - Same story when {ADDRESS, ZIP} is considered as a key.

  $\Rightarrow$ ADDR _INFO is in 3NF.

On the other hand, ADDR _INFO (CITY, ADDRESS, ZIP) has all the anomalies (insertion, deletion and modification).

- The FD {ZIP} $\rightarrow$ {CITY} is the real problem.

- Suppose, we delete the tuple

< Williamsburg, 55 Richmond Road, VA 23189>.

If this is the only 23189 tuple in the relation, we will lose the information

"VA 23189 is in Williamsburg".

$\Rightarrow$ *deletion anomaly.*

It is easy to check that insertion and modification anomalies are also present.

- The problem appears to be:
- In 2NF: we did not allow FD's from parts of keys to nonprime attributes.
- Here we have an FD from an attribute to <u>part of a key</u>.

- One option is to introduce the following rule for every relation:
  It should be 3NF
  There should be no FD X $\rightarrow$ Y such that Y is part of a key.

Unfortunately, this rule is too *restrictive.*

e.g., consider

LOTS_PRICE (PROP_ID, COUNTY, LOT#, PRICE).

Here,

- PROP_ID is the primary key.
- {COUNTY, LOT#} is another key.
- The FD {PROP_ID} $\rightarrow$ {COUNTY} follows.

Thus, if we disallow relations of this sort, we will be essentially barring all non-primary keys from having multiple attributes.

$\Rightarrow$ may be too restrictive in practice.

Let us try to soften the rule:
1. It should be in 3NF
2. For every FD X $\rightarrow$ Y, such that Y is part of a key, X should itself be a key.

That is, we do allow part of keys to be dependent on things – provided those things are keys.

This is a reasonable assumption because:

- If X is a key, we would likely have to check uniqueness anyway (and that's all we have to do).
- Deletion causes less of an anomaly, e.g., in
LOTS_PRICE (PROP_ID, COUNTY, LOT#, PRICE).

The FD {PROP_ID} $\rightarrow$ {COUNTY}is not important.

Deleting the only tuple with 'James City County', e.g.,

<169, James City, 23, $50,000>,

We lose the information "Prop_id 169 is in James City".

But, we wouldn't be deleting this if we weren't prepared to lose "169" forever.

$\Rightarrow$ it's OK to lose "Prop_id 169 is in James City".

The softened rule above needs a small modification:
Observe that in the above example, we have the FD

{PROP _ID} $\rightarrow$ {COUNTY},

This passes our new test.

However, the following is also an FD:

{PROP _ID,PRICE} $\rightarrow$ {COUNTY}.

This fails the test because {PROP_ID,PRICE} is not a key.

However, it is a superkey (contain a key).

Final form:

Definition. A relation R is in Boyce-Codd Normal Form (BCNF) if:

1. it is in 3NF
2. for every FD X$\rightarrow$Y such that Y is part of a key, X is a superkey.

3NF and BCNF: An Alternate Definition

- First recall the 3NF definition:
  1. 2NF
  2. no transitive dependency from a key to a nonprime attribute should exist.

Here, a transitive dependency means:

- X$\rightarrow$Y and Y $\rightarrow$ Z

- Y is not part of any key
- X is a key
- Z is a nonprime attribute

Now, since X is a key, the FD X→Y must be true for any Y.

Thus, the condition is really saying (given X is a key) that for Y → Z:

- (a) Y is not part of any key

- (b) Z is a nonprime attribute

Next, recall that 2NF is essentially:

- if Y→ Z then Y cannot be a proper subset of a key.

Combine this with the first item (a) in 3NF and write (b) separately:

A relation R is in 3NF if for every FD Y→ Z either

1. Y is a superkey, or
2. Z is a prime attribute.

- This is an alternate definition of 3NF which does not mention 2NF.
- Not that if Z is a prime attribute, we allow Y → Z even if Y is not a superkey, e.g. in
  ADDR-INFO (CITY, ADDRESS, ZIP)

We allowed {ZIP} → {CITY} because CITY is a prime attribute (it is apart of the key {CITY, ADDRESS}).

But BCNF does not allow this.

Hence, an alternate definition of BCNF is:

A relation R is in BCNF if for every FD Y → Z, Y is a superkey of R.

- This definition does not use 3NF.

Finally, observe that

R is in BCNF ⇒ R is in 3NF

⇒ R is in 2NF

⇒ R is in 1NF

4.6 MULTIVALUED DEPENDENCIES (MVD)

Definition: Given a relation scheme R, let X and Y be subsets of attributes of attributes of R (X and Y need not be distinct). Then the multivalued dependency denoted as X → → Y Satisfies in a relation R, if given two tuples t_1 and t_2 in R with t_1 (X) = t_2 (X); Where R (t_1,t_2,t_3,t_4) have the same X value. i.e.,

t_1 (X) = t_2 (X) = t_3 (X) = t_4 (X).

Whereas the Y values of t_1 and t_3 are same and the Y values of t_2 and t_4 are same, i.e.,

T1 (Y) = t3 (y) and t_2 (Y) = t_4 (Y)

And, the R-X-Y value of t_1 and t_4 are same

And the R-X-Y Values of t_2 and t_3 are same, i.e.,

t_1 (R-X-Y) = t_4 (R-X-Y)

t_2 (R-X-Y) = t_3 (R-X-Y)

Example: Illustration of MVD Definition is as follows:

X	Y	Z	- tuples
a	b_1	c_1	- tuple t_1
a	b_2	c_2	- tuple t_2
a	b_3	c_3	- tuple t_3
A	b_4	c_4	- tuple t_4

Therefore, for each X value in such a relation, there will be a set of Y values associated with it. This association between the x and Y values does not depend on the values of other attributes in the relation.

In the above example, we have two tuples t_1,t_2 in relation R defined on relation scheme R with the same X value (i.e., 'a'). We exchange the Y values (i.e., 'b_1& b_2') of these tuples t_1,t_2 and obtained the Y values of tuples t_3,t_4 (i.e., 'b_1 & b_2') [that is, same values as t_1,t_2]. Then tuples t_3 and t_4 must also be in R.

Thus, formally the above example is illustrated as MVD X $\rightarrow\rightarrow$ Y holds over R.

Given a set of FDs and MVDs, we can compute the several additional FDs and MVDs.
The first three rules are Armstrong's Axioms, which we saw earlier:

RULE 1: ***Reflexivity:*** If X $\supseteq$ Y, them X $\rightarrow$ Y.
RULE 2: ***Augmentation:*** If X $\rightarrow$ Y. then XZ $\rightarrow$ YZ for any Z.
RULE 3: ***Transitivity:*** If X $\rightarrow$ Y. and Y $\rightarrow$ Z then X $\rightarrow$ Z.
RULE 4: ***MVD Complementation:*** If X $\rightarrow\rightarrow$Y, then X $\rightarrow\rightarrow$ R - XY.
RULE 5: ***MVD Augmentation:*** If X $\rightarrow\rightarrow$ Y and W $\supseteq$ Z, then WX $\rightarrow\rightarrow$ YZ.
RULE 6: ***MVD Transitivity:*** If X $\rightarrow\rightarrow$ Y and Y $\rightarrow\rightarrow$ Z, then WX $\rightarrow\rightarrow$ YZ.
RULE 7: ***Replication:*** If X $\rightarrow$ Y, then X $\rightarrow\rightarrow$ Y.

RULE 8: ***Coalescence:*** If $X \rightarrow\rightarrow Y$ and there is a W such that $W \cap Y$ is empty, $W \rightarrow Z$, and $Y \supseteq Z$, then $X \rightarrow Z$.

Observe that in replication states, every FD is also an MVD.

4.7 JOIN DEPENDENCIES (JD)

A join dependency is a further generalization of MVD. **A join dependency (JD)** denoted as $(R_1,\ldots,R_n)$ is said to hold over a relation R if $R_1, \ldots,R_n$ is a lossless – join decomposition of R. Therefore, if $R = R_1 \ UR_2 \ U\ldots Urn$, We say that a relation r (R) satisfies the join dependency * $(R_1,R_2,\ldots R_n)$ if

$r = \Pi\, R_1\,(r) \bowtie \Pi\, R_2\,(r) \bowtie\bowtie \ \Pi\, R_n\,(r)$

A join dependency is **trivial** if one of the R_1 is R itself.

Example: Consider the join dependency * (R_1,R_2) on schema R. This dependency requires that, for all legal r(R)

$$r = \Pi\, R_1\,(r) \bowtie \Pi\, R_2\,(r)$$

Thus, $t_1\,[R_1 \cap R_2] = t_2\,[R_1 \cap r_2]$, but t_1 and t_2 have different values on all other attributes.

4.8 Properties of Decomposition

- We have seen that it is desirable to have relations in BCNF (or at least 3NF).
- We have seen how to test for BCNF and 3NF.
- But how do we create a BCNF (or 3NF) database?

One approach: Ad-hoc

* Create relations intuitively

* Test each for BCNF

More formal approach:

* Start with a single large relation with all attributes

* Systematically decompose relations not in GCNF

* Repeat until all relations are in BCNF

Unfortunately, decomposition can create problems:

- Dependencies may be lost after decomposition.
- Joins of decomposed relations may crate spurious tuples.

Thus far, we have identified problems with individual relations
We have not placed constraints among multiple relations.

3.8 Dependency Preservation

Suppose we decompose R = (A1,…An) into relations R1,…Rm.
Of course, we should have **attribute preservation,** i.e., attributes should not be lost in the shuffle:

$R_1 \cup R_2 \cup \ldots R_m = R$

Unfortunately, FD's can be lost, e.g.,

Suppose F is a set of FD's containing the dependency

{COUNTY} $\rightarrow$ {TAX _RATE}.

Suppose also that {COUNTY} is put in relation R_3 and {TAX _RATE} is put in relation R_7

we can't check the dependency.

In the above example, the lost FD would be OK, if the dependency were somehow not important

$\Rightarrow$ We need to consider the closure of FD's.

Definition. Let F be a set of FD's and suppose R is decomposed into relations $R_1, \ldots, R_m$. let G be the set of FD's in $R_1, \ldots, R_m$. Then the decomposition is **dependency preserving** if $G^+ = F^+$.

Fact: It is always possible to decompose any relation R into 3NF relations $R_1, \ldots, R_m$ such that the decomposition is dependency preserving.

(Proof is not easy).

Nonadditive Decompositions

Suppose we decompose R into $R_1, \ldots, R_m$. Later, we wish to recover R (perhaps as a view).

- The natural join on $R_1, \ldots, R_m$ should return R.
- If we're not careful, this join can create spurious tuples
  e.g., consider the relation r with schema R = (CAR, OWNER, COLOR):

CAR	OWNER	COLOR
Toyota	Smith	blue
Ford	Jones	blue

Suppose we decompose this into r_1 and r_2 with schemas R_1 = (CAR, COLOR) and R_2 = (OWNER, COLOR).

How? Let $r_1 = \Pi_{CAR, COLOR}(r)$ and $r_2 = \Pi_{OWNER, COLOR}(r)$:

CAR	COLOR
Toyota	blue
Ford	blue

OWNER	COLOR
Smith	blue
Jones	blue

What happens when we join r_1 and r_2?

CAR	OWNER	COLOR
Toyota	Smith	blue
Toyota	Jones	blue*
Ford	Smith	blue*
Ford	Jones	blue

$\Rightarrow$ Spurious tuples

A decomposition of R into R_1, …, R_m is **nonadditive** if for every instance r of R, the natural join of the corresponding instances R_1, …, R_m is equal to r, i.e.,

$r_1 * r_2 * \ldots * r_m = r$

where $r_i = \Pi R_i (r)$.

Note: **nonadditive** is the same as 'creates no spurious tuples'.
Also, called a **lossless** join.

Testing for Nonadditivity

- It would be useful, if given a decomposition, to test whether the decomposition is nonadditive.
- **A useful fact.** A decomposition of R into R_1 and R_2 is nonadditive with respect to a set of FD's F, if and only if either one of the FD's

  - $R_1 \cap R_2 \rightarrow R_1 - R_2$

  - $R_1 \cap R_2 \rightarrow R_2 - R_1$

  is in F^+.

  Institution:

  - Observe: the attributes $R_1 \cap R_2$ are in both R_1 and R_2
    $\Rightarrow$ these are the join attributes.

  - Suppose the FD $R_1 \cap R2 \rightarrow R_1 - R_2$ holds.
    T his is the same as $R_1 \cap R_2 \rightarrow R_1 - (R_1 \cap R2)$

    $\Rightarrow R_1 \cap R_2$ is a key for R_1.

    $\Rightarrow$ weird, unwanted tuple combinations can't occur.

In the R = (CAR, OWNER, COLOR) example:
We joined

CAR	COLOR	OWNER	COLOR

Toyota	blue	Smith	blue
Ford	blue	Jones	blue

to get

CAR	OWNER	COLOR
Toyota	Smith	blue
Toyota	Jones	blue*
Ford	Smith	blue*
Ford	Jones	blue

Note that {COLOR} is not a key for either relation.

Here, R_1 = (CAR,COLOR) and R_2 = (OWNER, COLOR)and,

$R_1 \cap R_2 = COLOR$

$R_1 - R_2 = CAR$

$R_1 - R_2 = OWNER$

Clearly, the neither of FD'S

{ COLOR } → {CAR}

{ COLOR } → {OWNER}

hold

⇒ the decomposition is not nonadditive.

Decomposition into Nonadditive BCNF Relations

Algorithm: Nonadditive BCNF decomposition

Input : Relation R, set of FD's F.

Output: A decomposition D = {R_1, R_2, …}

1. Set D ← {R};
2. **while** Ǝ non – BCNF relations in D **do**
3. Pick a non – BCNF relation Q from D
4. Find an FD X →Y in Q that violates GCNF
5. D ←D – {Q};
6. Create relation R' ← Q – Y;
7. Create relation R" ← X U Y;

Intuition: If X → Y is in some Q and it violates BCNF

$\Rightarrow$ X is not a superkey of Q (definition of BCNF)

$\Rightarrow$ we create a relation R" = X U Y

Here $X \rightarrow Y$ implies X is a superkey for R".

Also, we take out Y from Q (if not, it would violate BCNF).

Unfortunately, we can't always decompose R into BCNF relations that are both dependency preserving and nonadditive.

Fact. It is possible to decompose a relation into 3NF relations that are dependency preserving and nonadditive.

INCLUSION DEPENDENCIES

An inclusion dependency is a statement of the form that some columns of a relation are contained in other columns (usually of a second relation).

Example: A foreign key constraint is an example of an inclusion dependency; where, the referring column (s) of one relation must be as the primary key column (s)of the referenced relation.

The main point is that we should not split groups of attributes in an inclusion dependency.

Example: If we have an inclusion dependency AB $\subseteq$ CD, while decomposing the relation schema containing AB, we should ensure that at least one of the schemas obtained in the decomposition contains both A and B. otherwise, we cannot check the inclusion dependency AB $\subseteq$ CD without reconstructing the relation containing AB.

The inclusion dependencies mostly are **key-based** i.e., involve only keys. Foreign key constraints are a good example of key – based inclusion dependencies.

Fourth Normal Form (4NF)

Definition: Fourth normal form is a direct generalization of BCNF. Let R be a relation schema, X and Y be attributes of R, and F be a set of dependencies that includes both GDs and MVDs. Then R is said to be in **fourth normal form (4NF),** if, for every MVD $X \rightarrow\rightarrow Y$ that holds over, one of the following statements is true:

- $Y \subseteq X$ or XY = R, or
- X is a super key

Example: Consider a relation schema ABCD and suppose that the FDA $\rightarrow$ BCD and the MVD $B \rightarrow\rightarrow C$ given as follows:

B	C	A	D	tuples
b	c_1	a_1	d_1	- tuple t_1
b	c_2	a_2	d_2	- tuple t_2
b	c_3	a_3	d_3	- tuple t_3

The above figure shows three tuples from relation ABCD that satisfies the given MVD B $\rightarrow\rightarrow$ C. From the definition of an MVD, given tuples t_1 and t_2, it follows that tuple t_3 must also be included in the above relation. Now, consider tuples t_2 and t_3. From the given FD A $\rightarrow$ BCD and the fact that these tuples have the same A – value, we can compute that $C_1 = C_2$. Therefore, we see that the FD B $\rightarrow$ C must hold over ABCD whenever the FD A $\rightarrow$ BCD and the MVD B $\rightarrow\rightarrow$ C holds. If B $\rightarrow$ C holds, the relation is not in BCNF but the relation is in 4 NF.

Fifth Normal Form

Definition: A relation schema R is said to be in **fifth normal form (5NF)** if, foe every join dependency * (R_1,…R_n) that holds over R, one of the following statements is true:

- R_1 = R for some i, or
- The JD is implied by the set of those FDs over R in which the left side is a key for R.

Intuitively, we must be able to show that the decomposition of R into (R_1,…R_n) is lossless – join whenever the **key dependencies** (FDs in which the left side is a key for R) hold.

JD * (R_1,…R_n) is a **trivial JD** if Ri = R for some I, such a JD always hold.The following result helps us to safely ignore JD information.

- "If a relation schema is in 3NF and each of its keys consists of a single attribute then it is also in 5NF."
- The conditions identified in this result are sufficient for a relation to be in 5NF but they are not necessary.